

Name : Kritika Sharma
Section : 4

ASSIGNMENT - 1

Exercise 1: Student Attendance & Eligibility System A college tracks attendance for each subject. Attendance is captured daily as integers. At the end of the semester, attendance percentage is calculated as a double. University rules state that eligibility must be displayed as an integer percentage. Design logic to:

- Store raw attendance data
- Calculate percentage
- Convert the value safely for display
- Explain rounding vs truncation impact

Solution

```
static void Main()
{
    int totalClasses = 120;
    int attendedClasses = 89;

    double attendancePercentage = (double)attendedClasses / totalClasses * 100;
    int displayPercentage = (int)Math.Round(attendancePercentage);

    Console.WriteLine("Total Classes: " + totalClasses);
    Console.WriteLine("Attended Classes: " + attendedClasses);
    Console.WriteLine("Actual Percentage: " + attendancePercentage);
    Console.WriteLine("Displayed Percentage: " + displayPercentage + "%");

    if (displayPercentage >= 75)
    {
        Console.WriteLine("Student is eligible for exam");
    }
    else
    {
        Console.WriteLine("Student is not eligible for exam");
    }
}
```

Exercise 2: Online Examination Result Processing An online exam system stores marks per subject as int. The final average must be shown with two decimal places. Later, scholarship eligibility requires converting the average into an int. Design the conversion flow and explain precision loss scenarios.

```

static void Main()
{
    int s1 = 78;
    int s2 = 85;
    int s3 = 69;
    int s4 = 91;
    int s5 = 80;

    int totalMarks = s1 + s2 + s3 + s4 + s5;
    double average = (double)totalMarks / 5;
    Console.WriteLine("Average Marks: " + average.ToString("F2"));
    int scholarshipAverage = (int)Math.Round(average);

    Console.WriteLine("Average used for scholarship: " + scholarshipAverage);
}

```

Exercise 3: Library Fine Calculation System Fine per day is configured as decimal. Days overdue are stored as int. Total fine must be displayed as decimal but logged as double for analytics. Explain why different types are used and how conversions occur.

```

static void Main()
{
    decimal fine = 2.50m;
    int days = 7;
    decimal totalFine = fine * days;

    double totalFineForAnalytics = (double)totalFine;

    Console.WriteLine("Fine per day: " + fine);
    Console.WriteLine("Days overdue: " + days);
    Console.WriteLine("Total fine (for display): " + totalFine);
    Console.WriteLine("Total fine (for analytics): " + totalFineForAnalytics);
}

```

Exercise 4: Banking Interest Calculation Module Account balance is decimal. Interest rate is float from an external API. Monthly interest is calculated and added to balance. Demonstrate safe conversions and explain why implicit conversion may fail.

```

static void Main()
{
    decimal balance = 25000.75m;
    float interestRate = 1.2f;

    decimal monthlyInterest = balance * (decimal)(interestRate / 100f);
    balance = balance + monthlyInterest;

    Console.WriteLine("Interest added: " + monthlyInterest);
    Console.WriteLine("Updated balance: " + balance);
}

```

Exercise 5: E-Commerce Order Pricing Engine Cart total is accumulated using double. Tax and discount rules require decimal. Final payable amount is stored as decimal. Explain conversion strategy and precision risks.

```

static void Main()
{
    double cartTotal = 1234.56;
    decimal taxRate = 0.18m;
    decimal discount = 100m;

    decimal amount = (decimal)cartTotal;
    decimal tax = amount * taxRate;
    decimal finalAmount = amount + tax - discount;

    Console.WriteLine("Final payable amount: " + finalAmount);
}

```

Exercise 6: Weather Monitoring & Reporting Temperature sensors send readings as short. Values must be converted to Celsius and stored as double. Daily average is converted to int for dashboard display. Discuss overflow and casting concerns.

```

static void Main()
{
    short r1 = 300;
    short r2 = 320;
    short r3 = 310;

    double t1 = r1 / 10.0;
    double t2 = r2 / 10.0;
    double t3 = r3 / 10.0;

    double avg = (t1 + t2 + t3) / 3;
    int display = (int)Math.Round(avg);

    Console.WriteLine("Average temperature: " + display);
}

```

Exercise 7: University Grading Engine Final score is calculated as double. Grades are stored as byte values. Design logic to convert score to grade safely. Explain validation and casting choices.

```

static void Main()
{
    double score = 82.5;
    byte grade;

    if (score >= 90) grade = 10;
    else if (score >= 80) grade = 9;
    else if (score >= 70) grade = 8;
    else if (score >= 60) grade = 7;
    else grade = 6;

    Console.WriteLine("Grade: " + grade);
}

```

Exercise 8: Mobile Data Usage Tracker Usage is tracked in bytes as long. Displayed in MB and GB as double. Monthly summary rounds values to nearest integer. Explain implicit conversions and rounding methods.

```

static void Main()
{
    long bytesUsed = 5368709120;

    double mb = bytesUsed / (1024.0 * 1024.0);
    double gb = bytesUsed / (1024.0 * 1024.0 * 1024.0);

    int monthly = (int)Math.Round(gb);

    Console.WriteLine("Usage in MB: " + mb);
    Console.WriteLine("Usage in GB: " + gb);
    Console.WriteLine("Monthly summary: " + monthly + " GB");
}

```

Exercise 9: Warehouse Inventory Capacity Control Item count stored as int. Maximum capacity stored as ushort. Conversion required during comparison and reporting. Explain signed vs unsigned conversion risks.

```

static void Main()
{
    int itemCount = 450;
    ushort maxCapacity = 500;

    int limit = maxCapacity;

    if (itemCount > limit)
        Console.WriteLine("Capacity exceeded");
    else
        Console.WriteLine("Within capacity");
}

```

Exercise 10: Payroll Salary Computation Basic salary is int. Allowances and deductions are double. Net salary stored as decimal. Design type conversion flow and justify choices.

```

static void Main()
{
    int basic = 30000;
    double allowance = 4560.75;
    double deduction = 2100.50;

    double temp = basic + allowance - deduction;
    decimal netSalary = (decimal)temp;

    Console.WriteLine("Net Salary: " + netSalary);
}

```